

AI Product Comparison Engine

Technical Handover Document

Cross-encoder product matching with embedding retrieval. This document records the current state, every measurement taken, the reasoning behind each major decision, and what remains. It is written to be read cold, by someone with no prior context.

Status	Working end-to-end; benchmark-competitive; not production-hardened
Model	microsoft/deberta-v3-small cross-encoder, binary (same / different product)
Shipped config	v11 checkpoint, threshold 0.50, description budget 20w (S2.9)
Deploy bundle	deploy/ -- self-contained, 12/12 smoke + load-tested (S8)
Headline result	Beats published Ditto & RoBERTa baselines on 5 of 6 benchmarks
Generalisation	WDC-UNSEEN 75.52, best recorded (+4.38 over RoBERTa)
Real-market F1	80.85 on 190 hand-labelled Indian pairs (S1.5)
Retrieval	95.7% recall@50 over 112,746 products at 24 ms
Tests	54 collected: 53 passing, 1 xfail (documented, S4.1)
Date	14 August 2026

THE ONE THING TO KNOW. Every accuracy problem in this project turned out to be a data problem, never a model problem. The same architecture scored 45.9 F1 trained on rule-generated labels and 75.6 trained on human labels. Before changing the model, check the data.

THE SECOND THING. As of August 2026 the model is done for benchmark purposes. FOUR consecutive experiments returned null or noise-level results (S2.10). The remaining upside is real-world labelled data (S5.2) and production hardening (S5.1) -- not architecture, not more public corpora.

THE THIRD THING. Run-to-run variance on identical inputs is ~2 benchmark F1 and ~3.5 Indian F1 (S2.9). Do not draw a conclusion from a single training run; one nearly buried a working idea.

1. Results

1.1 Benchmark performance

One model trained once on a merged corpus, evaluated per benchmark. The published baselines used a separately trained model per benchmark, which makes matching them harder, not easier. 'argmax' is the honest number; 'best-thr' is tuned on the test set and is an optimistic upper bound.

Benchmark	argmax	best-thr	Published	Delta
Structured_Amazon-Google	78.57	79.12	Ditto 75.58	+2.99
Structured_Walmart-Amazon	84.38	85.64	Ditto 86.76	-2.38
Textual_Abt-Buy	89.36	90.28	Ditto 89.33	+0.03
WDC-SEEN	80.77	81.82	RoBERTa 78.58	+2.19
WDC-HALF-SEEN	80.51	81.50	RoBERTa 75.91	+4.60
WDC-UNSEEN	74.78	75.86	RoBERTa 71.14	+3.64

Caveat on three numbers. The Ditto ER-Magellan figures (75.58 / 86.76 / 89.33) are hardcoded in `train_on_real_corpus.py` from a secondary source and were never verified against the paper (the PDF fetch failed). The WDC / RoBERTa figures WERE verified from arxiv 2301.09521 Table 3 - and those are the ones beaten by the widest margins.

This table is NOT directly comparable to the calibration table in S2.8. Rescoring the same v7 checkpoint with `calibrate_threshold.py` gives Amazon-Google 77.89 (vs 78.57 here) and Walmart-Amazon 85.71 (vs 84.38), while WDC-HALF-SEEN and WDC-UNSEEN match to within 0.08. The pattern is unexplained: a different checkpoint would move all six, not four. Both tables are internally consistent - every model within a table was scored by the same code on the same data - so compare within a table, never across. Resolving this is open work.

1.1b Shipped model (v11) -- same evaluation, current checkpoint

Benchmark	argmax	best-thr	Published
Structured_Amazon-Google	76.74	77.05	Ditto 75.58
Structured_Walmart-Amazon	87.40	87.69	Ditto 86.76
Textual_Abt-Buy	91.73	91.93	Ditto 89.33
WDC-SEEN	82.54	82.70	RoBERTa 78.58
WDC-HALF-SEEN	79.62	81.82	RoBERTa 75.91
WDC-UNSEEN	75.52	76.16	RoBERTa 71.14
mean	82.26	82.89	-

v11 = 20-word description budget + hard-negative mined corpus (58,112 pairs). Beats the published baseline on 5 of 6, and WDC-UNSEEN 75.52 is the best figure this project has recorded.

1.2 Generalisation to unseen products

The drop from SEEN to UNSEEN measures whether a matcher handles products it has never encountered - the property that matters most for a live catalog.

System	SEEN -> UNSEEN drop
This model (v7 @ 0.50)	-6.73 (best)
v8, LSPC corpus, @ 0.50	-8.96
v8, LSPC corpus, calibrated @ 0.77	-7.68
RoBERTa (published)	-7.44
Ditto (published)	-8.92
R-SupCon (contrastive, published)	-24.65

R-SupCon has the best SEEN score of any published system and the worst UNSEEN score. This is direct evidence against adopting contrastive learning here. Note also that the LSPC corpus (v8) degrades this metric in every configuration, which is part of why v7 ships.

1.3 Retrieval at scale

Catalog	112,746 products
Index	FAISS IndexFlatIP (exact, no approximation), 165 MB
recall@1	25.7%
recall@10	75.4%
recall@50	95.7% <- top-50 is 0.04% of the catalog
recall@100	97.6%
Latency	24 ms p50 / 28 ms p95 (CPU)

recall@1 of 25.7% alongside recall@50 of 95.7% is the whole argument for two stages: the bi-encoder is a poor ranker but an excellent filter. Recall@k is the ceiling on the entire system - a match never retrieved is never re-ranked.

1.4 Real-world validation (Indian e-commerce)

14 pairs from live Amazon India / Google Shopping data, hand-labelled with written justifications. Small sample - directional, not a measurement.

Metric	Before serialization fix	After
Accuracy	79%	86%
Recall on true matches	0%	100%
Precision	n/a (no positives)	60% (3 TP of 5)

Also measured: on 137 cross-merchant pairs Google Shopping claims are the same product, the model agreed 85% of the time, and its disagreements were mostly Google's errors - independently reproducing hand labels on SanDisk Ultra vs CZ48, ColorFit Pulse 2 vs Icon 2, and Milton 900ml vs 520ml. On 150 same-brand different-model-number pairs the false-positive rate was 1%.

2. Key findings - read this before changing anything

These took days to establish. Each is a measurement, not an opinion, and each closes off a line of work that looks attractive but is already known to be wrong.

2.1 The labels were the bottleneck, not the model

The original dataset was generated by rules in `generate_relationship_pairs.py`. The model learned to reproduce the rule and scored 95% - a number that measured nothing. Graded against human-labelled WDC pairs the same checkpoint scored 45.9 F1. Retraining the identical architecture on 38k human labels gave 75.6. Nothing about the model changed.

2.2 Entity leakage barely mattered - and why that is informative

29.85% of test rows contained a product seen in training. Fixing it (entity-level split via union-find over the product graph) moved accuracy by 0.07 percentage points (0.9508 -> 0.9501). That should be impossible unless the model was never memorising entities. It wasn't: it was recovering the labelling rule, which generalises perfectly to unseen products. When labels come from a rule, leakage is nearly irrelevant.

2.3 A serialization mismatch silently destroyed all recall

The model is trained on `COL brand VAL ... COL title VAL ...`, but `inference.py` fed it `title | brand x | description`. Nothing raised. Three unmistakable matches scored 49.8% / 20.6% / 4.4% instead of ~100%, recall on real matches was zero, and `/health` reported 'ok' throughout. Now guarded: the serialization is recorded inside the checkpoint (`training_metadata.json` -> "serialization") and `ProductComparer` reads it, warning loudly when absent. The bi-encoder deliberately uses plain text instead - two models, two formats, both correct. Do not unify them.

2.4 WEAKLY_SIMILAR was never a real class

An audit of all 5,916 rows carrying that label found only 712 (12%) had any measurable justification, and just 2 came from a form-factor mismatch - both junk. 81% came from a per-category relative percentile rule; other rows came from an absolute Jaccard threshold and from a structural rule. Three incompatible definitions, one label. Attempts to build clean replacement data topped out at 438 rows with ~17% contamination. The class was dropped; the task is now binary.

2.5 Serialization format made no measurable difference to accuracy

A/B test of Ditto-style COL/VAL versus the project's own format: +0.10 F1 mean, with the sign flipping across splits. Within noise. (Separate from 2.3 - there the problem was train/inference mismatch, not the choice of format.)

2.6 Google Shopping clustering is not ground truth

It grouped 'Thirty First Earbuds For Boat 141' - a third-party item at roughly double the price - with the genuine boAt product, and placed one identical Amazon listing into two different colour clusters simultaneously. Measured disagreement with hand labels: 40% of its claimed matches. Useful as high-recall candidates; unusable as labels.

2.7 Public web-crawl data has saturated (LSPC null result)

WDC LSPC 2017 (`wdc/products-2017`) was merged to test whether more human-labelled pairs still help. It does not.

Leakage first. LSPC 2017 is an earlier crawl of the corpus WDC Products 2023 was built from, so contamination was the risk. Measured overlap against all three gold standards: **0.00%** at the strict comparator. Because a null result and a broken comparator look identical, the comparator was verified by feeding it LSPC against itself through the same code path (100.00%) and each gold standard against itself (100.00%). A containment probe - allowing a clean 2023 title to appear anywhere inside a noisier LSPC one - put the ceiling at 0.60% SEEN / 0.50% HALF-SEEN / **0.20% UNSEEN**. The corpus is clean.

Diversity, not volume, is the limit. LSPC xlarge is 171,787 train pairs over only ~13,812 distinct normalised titles - about 25 permutations per product. `--max-pairs` subsamples diversity-first (both-sides-unseen, then one-side-unseen, then fill) and reaches **100% product coverage in every category at 40,000 pairs**, ~3 pairs/product. The other 131k pairs are permutations of products already covered. Smaller `--size` values do not help: they are nested subsets of the same product pool.

Benchmark	v7 (38k)	v8 (+40k LSPC)	Delta
Structured_Amazon-Google	78.57	77.94	-0.63
Structured_Walmart-Amazon	84.38	87.86	+3.48
Textual_Abt-Buy	89.36	90.00	+0.64

WDC-SEEN	80.77	81.92	+1.15
WDC-HALF-SEEN	80.51	77.76	-2.75
WDC-UNSEEN	74.78	72.96	-1.82
mean	81.40	81.41	+0.01

Everything here is within noise. With ~500 positives on WDC splits and ~200 on ER-Magellan, F1 SE is roughly 1.5-2.5 points. The largest move (+3.48) is ~1.2 SE. Nothing clears the bar and the mean is flat. The finding is not 'LSPC hurt' - it is **40,000 more human-labelled pairs changed nothing measurable.**

Consequences. (a) Do not merge PriceRunner (~35k), ProMapEn (~1.5k) or similar - they would very likely land in the same place. (b) Do not chase LSPC V2020: it is built from PDC2020, the same corpus as the WDC Products 2023 gold standards, so it would contaminate the evaluation set - the SEVERE case, where UNSEEN goes UP while becoming meaningless. (c) v8 is kept as an artifact; v7 ships.

2.8 Threshold calibration also returned null; the shipped threshold is 0.50

v8's per-benchmark optimal thresholds were scattered (0.61 / 0.94 / 0.94 on the WDC splits), suggesting the argmax-at-0.5 decision rule was simply mis-set. `calibrate_threshold.py` fits ONE global threshold on validation only - never on test, unlike the 'best-thr' column - and applies it blind to every benchmark.

	v7	v8
fitted threshold	0.17	0.77
validation F1 @ 0.50	85.03	84.25
validation F1 @ fitted	85.34	84.66
validation gain	+0.32	+0.41
bootstrap validation F1 SE	0.82	~0.8
calibrated test mean	81.40	81.74
test mean @ 0.50	81.70	81.41
cost of not knowing domain	1.04	1.28
optimum spread (range / sd)	0.79 / 0.292	0.86 / 0.313

The minimum-improvement rule. Fitting always finds a threshold that beats the default on the data it fitted on - that is what argmax does, whether or not the improvement is real. Neither model's validation gain (+0.32, +0.41) clears one bootstrap F1 SE (0.82, from 1,113 positives). `calibrate_threshold.py` therefore adopts a fitted threshold only when it beats the default by at least --min-improvement-se SEs (default 1.0). Both models ship at 0.50.

This is decided on validation alone. Declining to move on sub-noise evidence is not test-tuning; it is refusing to act on a number that cannot be distinguished from zero. The rule also generalises to future checkpoints, which a one-off correction would not.

Why it matters that the rule exists. v7's validation curve was nearly flat and its per-benchmark optima bimodal (0.16 / 0.31 against 0.77 / 0.88 / 0.95), so the argmax landed at one extreme. Shipping the fitted 0.17 cost 0.30 mean F1 on test, including 1.00 on WDC-SEEN and 1.20 on WDC-HALF-SEEN. The gate would have prevented that.

Threshold spread is a selection criterion. Because one number must serve every domain, how far apart the per-benchmark optima sit matters independently of F1. A model whose optimum swings 0.08-0.94 cannot be served well by any single threshold. When two models tie on F1 within noise, prefer the tighter spread and the smaller cost-of-not-knowing - both of which favour v7.

Per-CATEGORY calibration remains viable if category is known at query time. Per-BENCHMARK thresholds are not servable: a live query does not arrive labelled 'Abt-Buy'.

2.9 Model selection: five checkpoints, and what separates them

Every checkpoint below is deberta-v3-small, binary, threshold 0.50 (the calibration gate kept the default on all five). They differ only in corpus and description budget.

	v7	v9 desc20	v10 hard	v11 hard
Benchmark mean	81.70	80.07	80.29	82.26

WDC-UNSEEN	74.78	-	72.23	75.52
Indian F1 (SE 3.6)	79.45	79.72	77.33	80.85
Indian precision	69.88	71.25	66.67	73.08
Indian errors	30	29	34	27
hard-slice precision	15.38	16.00	12.50	17.39
cost of not knowing	1.04	2.29	1.91	0.63
inference cost	1.00x	0.76x	0.76x	0.76x

v11 ships. Best on benchmarks, best on the target market, cheapest, and the best generalisation figure this project has recorded.

RUN-TO-RUN VARIANCE IS ~2 F1 AND IT INVALIDATED AN EARLIER VERDICT. v10 and v11 are the SAME recipe on the SAME corpus at the SAME seed (42), and differ by 2.0 benchmark F1 and 3.5 Indian F1. Seeding is not the whole story -- cuDNN autotuning and dataloader ordering are nondeterministic on GPU. On v10 alone the conclusion was 'hard-negative mining made things worse'; v11 overturned it. Any recipe comparison below ~2 F1 needs several runs. config.RANDOM_SEED now reads \$SEED so this is cheap to do.

2.10 Four experiments that returned null

Experiment	Result
LSPC 40k diversity-first merge	six-benchmark mean moved +0.01 (S2.7)
Threshold calibration	gate kept 0.50 on all five models (S2.8)
Description budget 100w -> 20w	-0.47 F1, half an SE; 24% fewer tokens. Kept for the speed, not accuracy
Hard-negative mining	hard slice 15.38 -> 17.39, inside run-to-run noise

The hard slice has never exceeded 17.39% precision across five checkpoints. That is the one number no amount of public data has moved, and the reason is now measured rather than guessed: of the 16,916 hardest negatives available in LSPC, only 4.7% are the model-suffix shape (VS104 vs VS104 Max) that Indian listings turn on. A 2017 European web crawl does not contain it. The remaining lever is ~2,000 hand-labelled Indian pairs, fine-tuned on top of v11 rather than trained from scratch -- 190 pairs measure a model, they cannot train one.

2.11 The model is order-sensitive; serving is not

"Is A the same product as B" is symmetric. A cross-encoder is not: [CLS] a [SEP] b [SEP] and its mirror are different inputs. dedupe() is order-invariant, so every pair reached training in exactly ONE order and the model never saw the mirror. Measured on the 190-pair Indian set: median gap between orders 0.06 pp, p95 35.9 pp, worst 91.8 pp, and 8 pairs (4.2%) flip across the threshold on argument order alone.

ProductComparer._canonical sorts the serialized texts, so both call orders become the same forward pass -- the SERVICE is now exactly order-invariant at zero cost. Averaging both directions was rejected: it doubles inference against a 2.3 req/s ceiling and is no more accurate (80.85 / 81.75 / 80.58 forward / reversed / averaged, all inside one SE). The MODEL is still asymmetric -- anything calling predict_probabilities directly sees the raw behaviour. build_real_corpus.py --swap-augment is the durable fix and is written but NOT yet trained.

2.12 Twelve contradictory labels, present since v7

dedupe() keys on (pair, LABEL), so a pair annotated both ways survives twice and the model is told one input is simultaneously a match and a non-match. 12 pairs, 24 rows, in every corpus built since v7. Both copies are now dropped -- there is no evidence which annotation is right, and a contradictory label is worse than a missing one.

Worth recording how nearly this went wrong. Two pandas implementations of the filter looked correct and were not: pd.Index over tuples silently builds a MultiIndex so groupby(level=0) grouped on the first text alone and flagged nothing; a "█"-joined key was truncated at the null byte, collapsing 18,526 pairs into 6,457 groups and inventing 1,026 contradictions that would have deleted 3,207 rows. It was caught only because the output was internally impossible -- 1,026 groups containing 1,031 rows, when each needs two. The shipped version is a plain dict checked against ground truth.

3. Architecture

```
POST /search {"product": {"title": "..."}}
|
+--> Stage 1 bi-encoder (all-MiniLM-L6-v2) + FAISS over full catalog
|     plain text | 24 ms | 95.7% recall@50
+--> Stage 2 cross-encoder (deberta-v3-small) re-ranks ~50 candidates
|     COL/VAL text | batched, one forward pass per batch
+--> exact_match + ranked alternatives
```

Neither stage works alone. The cross-encoder is too slow to score a catalog (~5-10 ms/pair on GPU means over an hour for 500k). The bi-encoder is too weak to pick the winner (25.7% recall@1). This retrieve-then-rerank split is the standard production architecture.

3.1 API surface

Endpoint	Purpose	Notes
POST /search	Find a product in the indexed catalog	Both stages. Needs an index; 503 if absent
POST /compare	Score a caller-supplied shortlist	No index needed. Max 200 candidates (413)
GET /health	Model + index status	Reports model_loaded, num_labels, catalog_size

3.2 Key files

File	Role
exact_match/inference.py	ProductComparer: compare(), score_pairs() (batched); reads serialization AND inference_threshold from the checkpoint
exact_match/train.py	Training loop (unmodified; drivers configure it from outside)
exact_match/dataset.py	Dynamic per-batch padding via DataCollatorWithPadding (training path only)
exact_match/preprocessing.py	Cleaning + ENTITY-LEVEL split (union-find)
ranking/embedding_retrieval.py	ProductIndex: build/save/load/search, recall@k
ranking/ranker.py	Two-stage orchestration; handles binary AND 5-class models
build_real_corpus.py	Merges WDC + ER-Magellan into one corpus, with leakage checks
train_on_real_corpus.py	Trains on the merged corpus; evaluates per benchmark
add_lspc_corpus.py	LSPC 2017 merge: leakage check, --max-pairs diversity-first subsampling; excludes LSPC from validation by default
calibrate_threshold.py	Fits the decision threshold on validation only, with the minimum-improvement gate
evaluate_on_wdc.py	Zero-shot evaluation against WDC gold standards
build_handover.py	Generates this document
scripts/manual/	0-assertion manual scripts. NOT tests - never move back to tests/

3.3 Training configuration that produced the shipped model (v7)

```
base          microsoft/deberta-v3-small (141.9M params)
corpus         38,145 human-labelled pairs (WDC 19,607 + Amazon-Google 6,674
               + Walmart-Amazon 6,142 + Abt-Buy 5,722)
labels        binary (2)
serialization  COL brand VAL ... COL title VAL ... COL description VAL ...
threshold     0.50 (see S2.8 - fitted values did not clear the noise floor)
lr 5e-5       effective batch 64 (32 x 2 accum)  warmup 0.05  fp16
epochs 15, early-stopped at 5 (best epoch 4)  ~40 min on a T4
```

4. Test results

46 collected, 45 passing, 1 xfail. All use stubs - no trained model or network required - so the suite runs in about 15 seconds.

Test file	N	Covers
-----------	---	--------

tests/test_split_leakage.py	7	Entity-level split; verified to FAIL against the old row-level split
tests/test_ranker_binary.py	8	Binary model path, batching, 413/503 handling
tests/test_search_endpoint.py	8	/search wiring, retrieve_k pass-through, failure modes
ranking/test_ranking.py	6	5-class ranking logic, category filtering, sort order
tests/test_api.py	3	/compare shape and validation
tests/test_data_quality.py	5	Contradiction rules, severity filtering
tests/test_generate_relationship_pairs.py	5	Labelling heuristic (1 xfail - see 4.1)
tests/test_utils.py	5	format_time

4.1 The known xfail

test_same_category_low_overlap_is_weakly_similar expects WEAKLY_SIMILAR for 'boAt Airdopes 300 / basic model' vs 'Sony WF earbuds / premium noise cancelling flagship'. It returns SIMILAR_ALTERNATIVE. The test asserts the Jaccard behaviour deliberately removed with the WEAKLY_SIMILAR class, but it documents a real gap: tier detection reads only numeric specs, so a tier stated in words is invisible. It is now marked xfail(strict=True) - adding lexical tier detection turns it into an XPASS failure that tells you to drop the marker. Previously it was a permanently red test, which trains people to ignore red tests.

4.2 Manual scripts (resolved)

tests/test_1.py, tests/test_example.py and root test.py had zero assertions, and test_1.py WROTE to data/ at import - so a bare pytest run silently rewrote a dataset. All three now live in scripts/manual/ with a README explaining why they must not go back.

5. What remains

5.1 Blocking real deployment

Item	Detail
No authentication	Verified: zero matches for <code>api_key</code> / <code>rate_limit</code> / <code>authenticate</code> in <code>api/</code> . Anyone reaching the port gets unlimited GPU inference. Do not expose publicly.
No rate limiting	<code>/compare caps</code> candidates at 200 and <code>/search caps retrieve_k</code> at 200, but there is no per-client limit.
Index cannot refresh	Rebuild is all-or-nothing, ~16 min for 112k on CPU. A live catalog needs incremental add/remove.
No Dockerfile	No deployment or container config exists.

5.2 Accuracy levers

Lever	Expected	Effort	Status
~2,000 Indian labelled pairs	Largest real gain	~2 sessions	OPEN - the only lever with evidence behind it. 193 candidates built, 14 labelled
deberta-v3-base	+1-3 F1	~1 day	Open; 3x params, untested
Per-category threshold calibration	unknown	~1 day	Open, needs category at query time
LSPC 219k corpus	+1-3 F1 (hoped)	-	DONE - null (S2.7)
Global threshold calibration	+1-2 F1 (hoped)	-	DONE - null (S2.8)
More public web-crawl corpora	-	-	CLOSED - saturated (S2.7)

5.3 Technical debt

Item	Detail
README is stale	163 lines with ZERO mentions of search, FAISS, retrieval, <code>trained_model_real</code> or <code>er_magellan</code> . Still documents a 2-class model on a 219-row toy dataset.
config.py defaults wrong	<code>RAW_DATA_PATH</code> still points at <code>products_structured.csv</code> . Harmless (drivers override) but misleading.
Dead synthetic generators	<code>generate_dataset.py</code> (683 lines, unreachable code), <code>generate_dataset_chunked.py</code> etc. still run and still write files. They produced the data that caused the problem.
<code>save_model.py</code> label_map bug	Writes a hardcoded binary label_map regardless of <code>NUM_LABELS</code> .
v7 checkpoint lacks 'serialization'	<code>trained_model_real_v7.zip</code> has no serialization field, so <code>ProductComparer</code> falls back to 'pipeline' and silently destroys recall (S2.3, trap 6.5). Fix before serving from it.
Large CSVs in git history	<code>data/relationship_pairs_final.csv</code> (57 MB) and ~200 MB of other tracked CSVs are output of the rule generator that produced the 45.9 F1 dead end. Removing them needs a history rewrite; judged not worth it.

5.4 Unvalidated

GPU latency (only CPU measured) | concurrent request handling (never load-tested) | catalog scale beyond 112,746 products | the S1.1-vs-S2.8 scoring discrepancy.

6. Traps that have already cost time

Each of these caused a real failure. None of them announce themselves.

6.1 config must be set BEFORE importing exact_match

model.py and save_model.py read config values as default argument values, which Python binds once at import. Setting config.NUM_LABELS afterwards is silently ignored and you get a 2-class model when you wanted 5. All driver scripts override config first, then import. ProductComparer now resolves its threshold per call rather than as a default argument, for exactly this reason.

6.2 .gitignore silently swallows files

*.txt swallowed all 41,519 ER-Magellan pairs; a negation rule (!data/er_magellan/**/*.*.txt) now exists. An old /api rule left api/schemas.py untracked, so a fresh clone could not import the API. Separately, a printf >> append without a trailing newline concatenated '.env' and 'trained_model_real/' into one dead rule, leaving a 549 MB checkpoint committable. Checkpoints are now globbed (trained_model*/). After editing this file, verify with git check-ignore -v.

6.3 Kaggle decompresses uploaded archives

Files uploaded as .json.gz arrive as .json, and zipped checkpoints arrive as extracted directories. Loaders must accept both. /kaggle/working is wiped between sessions and /kaggle/input is read-only - copy checkpoints into /kaggle/working before writing to them, and save anything you want to keep before the session ends.

6.4 tqdm floods Kaggle logs

Under !python the progress bar cannot rewrite its line, so every step prints a new row and buries the results. Driver scripts disable it by default (--progress re-enables).

6.5 Zip the right model directory

trained_model/ was the OLD 5-class model (45.9 F1). The good v7 checkpoint is trained_model_real/. A notebook cell reading make_archive(..., 'trained_model') while training exported to trained_model_real/ is how a wrong-directory archive gets created. Always zip the directory the training script actually wrote to.

6.6 A 5-class checkpoint scored as binary produces plausible nonsense

Reading probs[:, 1] as P(match) on a 5-class model returns P(SAME_PRODUCT_DIFFERENT_VARIANT). Nothing raises. Measured: validation F1 13.20 and a fitted threshold of 0.05 - the bottom of the sweep, the signature of a noise score column - reported as though they were results. calibrate_threshold.py now hard-fails on num_labels != 2, locates the match column by name, and stops if the best validation F1 is under 0.40.

6.7 datasets >= 4.0 refuses script-based HuggingFace datasets

wdc/products-2017 ships products-2017.py, so load_dataset() fails for every config with 'Dataset scripts are no longer supported' - which both call sites reported as 'check internet access'. Both now fetch the plain .json.gz files directly via hf_hub_download, removing the datasets dependency from those paths entirely. Note train/valid are size-partitioned but test is NOT (one test.json.gz per category).

6.8 Fit thresholds on a validation set that covers every domain

A stale 6,105-pair validation set missing all WDC-Products rows produced a fitted threshold of 0.17 - essentially Amazon-Google's own optimum of 0.16 - and cost WDC-SEEN 1.00 F1. calibrate_threshold.py now prints validation composition and warns when a benchmark domain is absent.

7. Reproducing the shipped model

```
# 1. build the corpus (needs data/er_magellan + WDC 50pair files)
python build_real_corpus.py --wdc /path/to/wdc --wdc-size large

# 2. train (~40 min on a T4)
python train_on_real_corpus.py --wdc /path/to/wdc

# 3. confirm the decision threshold (expect: KEPT 0.50)
python calibrate_threshold.py --model trained_model_real --wdc /path/to/wdc --write

# 4. build the retrieval index
python -m ranking.embedding_retrieval build --catalog <catalog.jsonl> \
    --out data/product_index

# 5. measure recall BEFORE trusting it
python -m ranking.embedding_retrieval evaluate-pairs --index data/product_index \
```

```
--ground-truth data/scale_ground_truth.csv
```

```
# 6. serve
uvicorn api.main:app --host 0.0.0.0 --port 8000
```

Step 3 is optional for reproduction but required for any new checkpoint: it records `inference_threshold` inside the model directory, which `ProductComparer` reads. `add_lspc_corpus.py` is deliberately NOT in this sequence - see S2.7.

Data sources (all free, human-labelled)

Source	Pairs	Where
WDC Products 2023	~20k train + 3 x 4,500 gold	webdatacommons.org/largescaleproductcorpus/wdc-products/
ER-Magellan (Abt-Buy, Amazon-Google, Walmart-Amazon)	31,277	github.com/megagonlabs/ditto -> data/er_magellan/
WDC LSPC 2017	219,135 at xlarge	huggingface.co/datasets/wdc/products-2017 (merged, null result - S2.7)

End of handover document. Generated 14 August 2026 by `build_handover.py`.